Video Anomaly Detection Complete Project Journey

Decisions, challenges, experiments, hyperparameters, and tuning

Dataset: ShanghaiTech Campus / CUHK Avenue / UCSD Ped

Framework: Anomalib 2.7.0, PyTorch 2.11.0+cu128

Hardware: NVIDIA RTX 3060 12 GB, Windows

Date: 2026-11-02

Complete Project Journey

Complete Project Journey — Video Anomaly Detection

Project: Unsupervised video anomaly detection on surveillance benchmarks **Workspace:** `F:\Developer Stack\ML vision` **Date:** 2026-11-02 **Hardware:** NVIDIA RTX 3060 12 GB, 7.9 GB system RAM, Windows **Stack:** Python 3.10.11, PyTorch 2.11.0+cu128, Anomalib 2.7.0, Gradio 6.31.0

This document records the full journey: how the idea changed, every major decision, the environment built, the datasets acquired, the failures and workarounds, the first experiments, hyperparameters used, and what still needs tuning.

Companion documents:

- `docs/DECISIONS.md` — decision records (ADR style)
- `docs/CHALLENGES.md` — problem catalog with root causes and fixes
- `docs/EXPERIMENTS.md` — experiment log, hyperparameters, tuning plan
- `PROJECT_GUIDE.md` — reference manual and roadmap
- `docs/PROGRESS.md` — short running status log

---

0. Executive summary

The project started as an industrial image anomaly-detection idea (PaDiM / PatchCore / WinCLIP on MVTec-AD), then changed to **video anomaly detection** because the user wanted video and MVTec-AD contains only still images. The final scope is surveillance video anomaly detection on **ShanghaiTech Campus**, with **CUHK Avenue** and **UCSD Ped1/Ped2** as secondary datasets.

The environment was built from scratch: a Python 3.10 venv, CUDA PyTorch, Anomalib 2.7.0, Gradio, PyAV, OpenCLIP, and TorchCodec. A non-obvious FFmpeg shared-library fix was required before any video could be decoded. All three datasets were acquired and prepared; ShanghaiTech was ultimately fetched from a Hugging Face mirror after the official OneDrive route blocked automated downloads.

One real experiment now exists: **PatchCore on ShanghaiTech scene 1**, with **image AUROC 0.7853**, **image F1 0.6716**, **pixel AUROC 0.9887**, and **pixel F1 0.5948** under a deliberately subsampled protocol. The subsampling is required because the machine has only 7.9 GB of RAM: a full test ran inference but ran out of memory while computing pixel AUROC.

The project is therefore at the transition from "foundation ready" to "systematic benchmarking and tuning".

---

1. How the idea evolved

1.1 Original ask

The initial request was a computer-vision project that:

- uses current technology,
- is not too hard,
- demonstrates a strong ML/vision foundation,
- is suitable for a portfolio or interview.

Several ideas were proposed: text-promptable segmentation with Grounding DINO + SAM 2, real-time detection with YOLO + tracking, monocular depth with Depth Anything V2, image retrieval with CLIP, and **industrial anomaly detection**.

1.2 First choice: industrial image anomaly detection

The industrial anomaly-detection idea was selected and expanded:

- Datasets: **MVTec-AD**, 15 categories.
- Methods: **PaDiM** (classical), **PatchCore** (modern), **WinCLIP** (zero-shot).
- Metrics: image/pixel AUROC, PRO, F1, runtime.
- Library: **Anomalib**.
- Scope: 3 categories first, then all 15.
- Demo: **Gradio**.
- Ordering: WinCLIP last.

1.3 The pivot to video

The user then said: **"i want video"**.

Verification against the official sources and the installed Anomalib source showed:

- **MVTec-AD has no video** — it is a still-image dataset.
- **MVTec AD 2** (the newer benchmark) is also still images: 8,000+ high-resolution images, not clips.
- Anomalib 2.7 supports video datamodules: `Avenue`, `ShanghaiTech`, `UCSDped`.
- Anomalib 2.7 also contains video-native models: `AiVad`, `Fuvas`.

The user accepted surveillance video (as opposed to insisting on an industrial video benchmark). The project became:

> Train detectors on normal surveillance footage only, then detect and localize abnormal events in test video, comparing frame-wise, video-native, few-shot, and zero-shot methods.

1.4 Final project definition

Element	Final choice
Task	Unsupervised video anomaly detection
Primary dataset	ShanghaiTech Campus (13 scenes)
Secondary datasets	CUHK Avenue, UCSD Ped1/Ped2
Framework	Anomalib 2.7.0
Models	PaDiM, PatchCore, AI-VAD, FUVAS, WinCLIP

Element	Final choice
First scope	3 scenes, then all 13
Demo	Gradio
Python	Dedicated 3.10 venv
Output	Metrics table, heatmaps, timelines, annotated video, report

---

2. Environment and hardware discovery

2.1 Machine facts

Item	Value
GPU	NVIDIA GeForce RTX 3060, 12,288 MiB
Driver	621.01
CUDA UMD	13.5
OS	Windows, PowerShell 5.1
Default Python	3.13.9
Available Python	3.10.11
System RAM	**7.9 GB total**, about 5.8 GB free
Disk C:	41.8 GB free at audit time
Disk F:	69 GB free at audit time; project lives here

2.2 Initial package state (system Python 3.13)

Package	Version
torch	2.10.0+cpu
torchvision	0.25.0
numpy	2.3.5
scipy	1.16.3
scikit-learn	1.8.0
opencv-python	4.13.0.92
pandas	3.0.4
pillow	12.0.0
matplotlib	3.10.8
tqdm	4.67.1
transformers	4.57.6

Package	Version
onnxruntime	1.31.0
rapidocr-onnxruntime	1.2.3

Missing: Anomalib, Gradio, timm, Lightning, av, TorchCodec, OpenCLIP.

Tools: `tar` present, `7z` absent, `conda` absent, `uv` absent, `winget` present. Chrome and Edge installed.

2.3 The critical discovery: 7.9 GB RAM

The GPU is comfortable (12 GB VRAM), but system RAM is small. This later became the single biggest constraint for evaluation. GPU memory was never the limiting factor; CPU RAM was.

---

3. Tooling decisions and why

3.1 Anomalib instead of from-scratch implementations

The user chose the library route. Consequences:

- Fast access to PaDiM, PatchCore, AI-VAD, FUVAS, WinCLIP.
- Built-in metrics, post-processing, thresholding, checkpointing.
- Less "from scratch" signal in interviews, compensated by rigorous experiments, ablations, and an honest analysis of library behavior.
- Library-specific issues must be documented (visualizer bug, video batch types, TorchCodec dependency).

3.2 Python 3.10 venv

Anomalib 2.7.0 metadata says `requires_python >=3.10`, but its published classifiers only cover 3.10, 3.11, and 3.12. The system Python is 3.13.9, outside the tested range. Python 3.10.11 was already installed, so a `.venv` was created with it.

3.3 CUDA 12.8 wheels

The system PyTorch was CPU-only. The CUDA build was installed from `https://download.pytorch.org/whl/cu128` first, before Anomalib, to reduce the chance of Anomalib or Lightning pulling a CPU wheel. The resulting versions were newer than planned: **torch 2.11.0+cu128** and **torchvision 0.26.0+cu128**.

3.4 Install the video extras explicitly

Anomalib did not bring `av`, `open-clip-torch`, or `torchcodec` automatically. They were installed explicitly:

```
.\.venv\Scripts\python.exe -m pip install av "open-clip-torch" torchcodec
```

3.5 Install order

1. Python 3.10 venv.

2. Upgrade pip.
3. CUDA torch/torchvision.
4. Verify CUDA.
5. Anomalib + Gradio.
6. Verify that Anomalib did not replace torch.
7. Video extras.
8. FFmpeg shared DLLs.

---

4. Dataset acquisition

4.1 CUHK Avenue

- Official page: `http://www.cse.cuhk.edu.hk/leojia/projects/detectabnormal/dataset.html`
- Videos: `Avenue_Dataset.zip` (813,227,845 bytes ≈ 775.6 MB).
- Ground truth: `ground_truth_demo.zip` (931,269 bytes ≈ 0.9 MB).
- Contents: 16 training videos (normal), 21 test videos (normal + abnormal), 47 abnormal events.
- Download method: `curl -L --fail --retry 3`.
- Extraction: `tar -xf` (Windows bsdtar handles zip).

The archive extracted a top-level `Avenue Dataset/` folder. Anomalib expects `training_videos/` and `testing_videos/` immediately under the root, so those folders were moved up one level. `ground_truth_demo/` stayed at the root because that is Anomalib's default `gt_dir`.

4.2 UCSD Ped1 / Ped2

- Official page: `http://www.svcl.ucsd.edu/projects/anomaly/dataset.htm`
- Archive: `UCSD_Anomaly_Dataset.tar.gz` (740,306,953 bytes ≈ 706 MB).
- Contents: Ped1 (34 train / 36 test) and Ped2 (16 train / 12 test).
- Videos are stored as folders of frames, not single video files.
- Some test folders have `*_gt/` subfolders with ground-truth masks.
- Extracted to `data/ucsdped/UCSD_Anomaly_Dataset.v1p2/`.

4.3 ShanghaiTech Campus — the difficult one

Official page: `https://svip-lab.github.io/dataset/campus_dataset.html`

The official distribution is a folder share on OneDrive:

```
https://1drv.ms/u/s!AjjUqiJZsj8whLt-1ABerTT-9eH9Ag?e=eJbY6Y
```

Inside: `OpenSources > ano_pred_cvpr_2018 > ShanghaiTechCampus > splits`, containing 7 parts of one archive plus a readme.

What went wrong

1. **Direct API access failed.** - `api.onedrive.com/.../shares/u!...` returned HTTP 400. - The short link returned 301 to `onedrive.live.com`, then **403 Forbidden**.
2. **Chrome browser download started but vanished.** - Chrome wrote `data\Unconfirmed 267844.crdownload`. - It reached at least ~904 MB (measured from actual file bytes; `Get-ChildItem` showed stale sizes such as 11.1 MB because Chrome held the file open). - At one point the temp file disappeared entirely and no `splits.zip` existed anywhere.
3. **Free Download Manager could not handle it.** - Pasting the folder share link produced: "Server has returned a web page. Click to open it in the web browser." - This is expected: the link is a web page, not a direct file. - Right-clicking the OneDrive Download button offered no "copy link".
4. **Chrome's download page trick was plausible but not used in the end** because the download became unreliable.

What worked

A Hugging Face mirror of the same archive:

```
Dataset: 123zdm/ShanghaiTech
Files:   shanghaitech.tar.gz.aa ... shanghaitech.tar.gz.ag
```

It was downloaded with `huggingface_hub` (already installed as an Anomalib dependency):

```
from huggingface_hub import hf_hub_download
for s in ["aa","ab","ac","ad","ae","af","ag"]:
    hf_hub_download(repo_id="123zdm/ShanghaiTech",
                    filename=f"shanghaitech.tar.gz.{s}",
                    repo_type="dataset",
                    local_dir="data/shanghaitech_parts")
```

Integrity verification

Part	Bytes
`.aa` – `.af`	1,073,741,824 each
`.ag`	550,511,474
Total	**6,992,962,418 (6.51 GB)**

The parts were joined with `cmd /c copy /b`. The joined file length matched the expected total byte-for-byte. The archive was extracted with `tar -xzf`.

Layout after extraction

The archive extracted into a nested `shanghaitech/` folder. Its contents were moved up so that `training/` and `testing/` are immediate children of `data/shanghaitech/`.

Actual contents:

Path	Count	Notes
`training/videos/`	330 `.avi`	normal clips
`testing/frames/`	107 folders	`.jpg` frames per test clip

Path	Count	Notes
`testing/test_frame_mask/`	107 files	frame-level labels
`testing/test_pixel_mask/`	107 `.npy` files	pixel-level masks

There was **no** `training/frames/` in the archive, even though some documentation implies it. Anomalib's ShanghaiTech dataset class uses `training/converted_videos/*.avi` for training and `testing/frames/<clip>/*.jpg` for testing.

Encoding problem and re-encode

The raw ShanghaiTech videos have encoding quirks. The first attempt was to hardlink the raw videos into `training/converted_videos/` and skip Anomalib's re-encode, because TorchCodec could decode most of them. This failed during training with:

```
RuntimeError: Requested next frame while there are no more frames left to decode.
```

The hardlinks were removed, and Anomalib's intended `prepare_data()` path was run. It re-encoded all 330 training videos to XVID. Output: 330 files, about **2.28 GB**, produced over roughly 15–25 minutes. After that, frame indexing and training were stable.

Disk footprint

ShanghaiTech on disk is large because it contains the 7 parts, the joined archive, the extracted raw data, and the re-encoded videos:

```
data/shanghaitech (total)        ~24.4 GB
```

This is acceptable on F: (about 86 GB free at the time of writing), but the 7 parts and the joined archive are duplicates and can be deleted after a verified backup/extraction.

---

5. Environment build details

5.1 Commands that built the environment

```
py -3.10 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip

.\.venv\Scripts\python.exe -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu128
.\.venv\Scripts\python.exe -m pip install "anomalib==2.7.0" gradio
.\.venv\Scripts\python.exe -m pip install av "open-clip-torch" torchcodec
```

5.2 Verified versions

Package	Version
Python	3.10.11
pip	26.2.2
torch	2.11.0+cu128
torchvision	0.26.0+cu128
anomalib	2.7.0
gradio	6.31.0
pytorch-lightning	2.6.6
lightning	2.6.6
timm	1.0.30
torchmetrics	1.9.0
av	17.1.0
torchcodec	0.16.0
open-clip-torch	3.3.0
opencv-python-headless	5.0.0.93
scikit-learn	1.7.2
scipy	1.15.3
numpy	2.2.6
pandas	2.3.3
matplotlib	3.10.9
tqdm	4.70.1

5.3 GPU verification

```
import torch
print(torch.__version__)                # 2.11.0+cu128
print(torch.cuda.is_available())        # True
print(torch.cuda.get_device_name(0))    # NVIDIA GeForce RTX 3060
```

5.4 Model and data API verification

All of the following imported successfully:

```
from anomalib.data import ShanghaiTech, Avenue, UCSDped
from anomalib.models import Padim, Patchcore, AiVad, Fuvas, WinClip
from anomalib.engine import Engine
```

Verified constructor parameters (excerpt):

- `ShanghaiTech(root, scene, clip_length_in_frames, frames_between_clips, target_frame, ...)`

- `Avenue(root, gt_dir, clip_length_in_frames, frames_between_clips, ...)`
- `UCSDped(root, category, clip_length_in_frames, frames_between_clips, ...)`
- `Patchcore(backbone, layers, pre_trained, coreset_sampling_ratio, num_neighbors, precision, ...)`
- `Padim(backbone, layers, pre_trained, n_features, ...)`
- `AiVad(box_score_thresh, persons_only, min_bbox_area, max_bbox_overlap, enable_foreground_detections, n_velocity_bins, use_velocity_features, use_pose_features, use_deep_features, ...)`
- `Fuvas(backbone, layer, pre_trained, spatial_pool, pooling_kernel_size, pca_level, ...)`
- `WinClip(class_name, k_shot, scales, few_shot_source, ...)`

---

6. The video-decoding rabbit hole

6.1 Symptom

The first Avenue datamodule setup failed with:

```
ImportError: Video decoding capabilities were removed from torchvision and migrated to TorchCodec.
```

torchvision 0.26 no longer decodes video itself.

6.2 Installing TorchCodec

```
.\.venv\Scripts\python.exe -m pip install torchcodec
```

Installed: `torchcodec 0.16.0+cpu`.

6.3 TorchCodec still failed

```
RuntimeError: Could not load libtorchcodec. Likely causes:
1. FFmpeg is not properly installed in your environment...
On Windows, ensure you've installed the "full-shared" version which ships DLLs.
```

No FFmpeg was installed on the machine. TorchCodec needs **shared** FFmpeg libraries (`avcodec-*.dll`, `avformat-*.dll`, etc.), not just a static `ffmpeg.exe`.

6.4 Portable FFmpeg shared build

Downloaded:

```
https://github.com/BtbN/FFmpeg-Builds/releases/download/latest/ffmpeg-master-latest-win64-gpl-shared.zip
```

Size: 90,941,653 bytes. Extracted to:

```
tools/ffmpeg/ffmpeg-master-latest-win64-gpl-shared/bin
```

It contained FFmpeg 9 libraries, including `avcodec-63.dll`.

6.5 The DLL-location trap

First attempt: prepend the FFmpeg `bin` to `PATH` for the process. This worked for a manual decode test.

Second attempt: copy the DLLs into `.venv\Scripts\` so the app directory would find them. This **did not** work; TorchCodec still failed to load.

Third attempt (successful): copy the DLLs into the folder that contains the TorchCodec extension DLLs:

```
Copy-Item "$ffmpegBin\*.dll" ".venv\Lib\site-packages\torchcodec\" -Force
```

Windows resolves dependent DLLs from the directory of the loaded DLL, so this made `libtorchcodec_core9.dll` find `avcodec-63.dll` and friends.

Verified:

```
from torchcodec.decoders import VideoDecoder
d = VideoDecoder("data/avenue/training_videos/01.avi")
print(d.metadata.num_frames, d[0].shape)   # 1364, (3, 360, 640)
```

---

7. Data preparation details

7.1 Avenue

Final layout:

```
data/avenue/
├── training_videos/        # 16 normal clips
├── testing_videos/         # 21 clips
├── training_vol/
├── testing_vol/
├── ground_truth_demo/      # masks, converted from .mat to .png by Anomalib
├── Avenue_Dataset.zip
└── ground_truth_demo.zip
```

Datamodule test result at `clip_length_in_frames=1`:

- train samples: **15,328**
- test samples: **15,324**
- batch image shape: `(B, 3, 480, 856)`

7.2 ShanghaiTech

Scene 1 at `clip_length_in_frames=1`, `frames_between_clips=1`:

- train clips: **69,976**
- test clips: **11,894**
- batch image shape: `(8, 3, 480, 856)`

Scene 1 at `frames_between_clips=100`:

- train clips: **47 batches** at batch size 16, i.e. about **700–750 frames**.

7.3 Why frame sampling is mandatory for PatchCore

PatchCore stores a feature vector for every patch of every training image.

Approximate calculation for ShanghaiTech scene 1:

- 69,976 frames
- 256×256 input
- WideResNet-50 layer2 stride 8 → 32×32 = 1,024 patches per frame
- concatenated channels: 512 (layer2) + 1,024 (layer3) = 1,536
- storage: 69,976 × 1,024 × 1,536 × 4 bytes ≈ **440 GB**

That is impossible on this machine. The memory bank must be built from a sampled subset of normal frames (or from lower-resolution/feature-reduced representations). The first real experiment used every 100th normal frame (~700 frames).

---

8. First smoke tests

8.1 PatchCore smoke fit (5 batches)

Configuration:

- scene 1
- `clip_length_in_frames=1`
- batch size 8
- `limit_train_batches=5`, `limit_val_batches=2`, `limit_test_batches=2`
- PatchCore, WideResNet-50, layers 2+3, coreset 0.01, neighbors 9

Result:

- GPU used: **True** (Lightning logged `used: True`, device RTX 3060).
- Fit completed, coreset selected over 408 candidate points.
- Test crashed in the visualizer:

```
AttributeError: 'VideoItem' object has no attribute 'image_path'
```

Root cause: Anomalib's `ImageVisualizer` assumes image datasets with `image_path`. Video datasets produce `VideoItem` with `video_path`. Metrics are not affected.

Fix: pass `visualizer=False` when using image models on video data.

8.2 Smoke test after the fix

- Fit + test completed.
- Metrics were 0.0 because the limited test subset contained only normal frames ("No positive samples in targets"). Expected for a 2-batch smoke test, not a bug.

9. First real experiment

9.1 Protocol

Item	Value
Model	PatchCore
Backbone	WideResNet-50 (`wide_resnet50_2`)
Layers	`layer2` , `layer3`
Input	256×256, ImageNet normalization
Coreset ratio	0.01
Neighbors	9
Precision	float32
Train sampling	every 100th normal frame (~700 frames)
Test sampling	every 10th test frame (1,204 frames)
Batch size	16
Workers	0
Max epochs	1 (feature fit, no gradient training)
Threshold	adaptive, from validation split
Visualizer	disabled

9.2 First attempt with full test

- Fit: **239.5 s**
- Test inference: **744 batches / 11,894 frames**, **33 min 52 s**
- Then AUROC computation crashed:

```
DefaultCPUAllocator: not enough memory: you tried to allocate 6,235,881,472 bytes
```

Cause: TorchMetrics accumulated pixel predictions for all frames and tried to sort them. With 11,894 frames and 256×256 maps, that is about 780 million values.

9.3 Re-test from checkpoint with stride 10

The fit checkpoint had been saved:

```
results/patchcore/scene01/Patchcore/ShanghaiTech/v0/weights/lightning/model.ckpt
```

It was reloaded with `Patchcore.load_from_checkpoint(...)` , and the test datamodule was created with `frames_between_clips=10` (1,204 test frames).

Result:

Metric	Value
image_AUROC	**0.7853**
image_F1Score	**0.6716**
pixel_AUROC	**0.9887**
pixel_F1Score	**0.5948**

Test time: **379.9 s** (about 3.2 frames/s end-to-end including metric computation).

9.4 Interpretation

- Pixel localization is strong (0.989 pixel AUROC).
- Image-level detection is moderate (0.785 image AUROC).
- This is plausible for a frame-wise image model on ShanghaiTech: motion anomalies such as running/chasing are hard without temporal reasoning. This motivates AI-VAD and FUVAS.
- Numbers are not directly comparable to published video results because the test set was subsampled and the memory bank is small.

9.5 First annotated demo video

A renderer was written (`src/render_video.py`) to turn the fitted PatchCore checkpoint into a watchable MP4. It runs frame-by-frame inference, thresholds the anomaly map, finds contours, and draws red boxes with a score banner.

Source clip: **`01_0136`** (scene 1, 529 frames, 856×480, 311 ground-truth anomalous frames).

The first render used the checkpoint thresholds (pixel 48.84, image 46.27). It produced many small boxes on normal frames (for example a backpack at frame 62/80), so a threshold sweep was done against the clip's frame mask:

Image threshold	Precision	Recall	F1
46.3 (checkpoint)	0.66	1.00	0.80
52	0.89	0.93	0.91
54	0.97	0.89	0.93
58	0.99	0.79	0.88

Final display settings: **pixel threshold 55**, **image threshold 58**, **min box area 200**, boxes drawn only when the frame is labelled ANOMALY. This removed the small false boxes while keeping the truck. The two remaining high-score frames with GT=0 (271, 323) visibly contain the truck, so they are likely sparse annotation rather than model errors.

Result:

- 529 frames rendered in 19.8 s (~26.7 fps)
- 247 ANOMALY frames, 245 with boxes
- first/last boxed frame: 169 / 431
- max score 82.35 (truck)
- 856×480 @ 10 fps, 4.4 MB

Artifacts:

```
results/videos/shanghaitech_scene01_01_0136_annotated.mp4
results/videos/shanghaitech_scene01_01_0136_annotated.scores.csv
results/videos/01_0136_samples/*.jpg
```

This is a visualization-only result; it does not change the metric protocol. The display thresholds are tuned for presentation, while reported metrics use the checkpoint thresholds.

9.6 People-only AI-VAD demo

The first demo video was a truck (PatchCore). The user then asked for a **people-only** video. Testing the fight clip `01_0026` with PatchCore gave scores of 44-56 and **no boxes**; a scan of all scene-1 clips showed the frame-wise model only fires on vehicles. This is the motion-blindness the project anticipated.

The video-native **AI-VAD** was therefore fitted with `persons_only=True`:

- Model: Mask R-CNN person detection + Keypoint R-CNN + RAFT flow + CLIP ViT-B/16 + velocity/pose/deep density estimators.
- Data: scene 1, clips of 2 frames, every 100th frame (738 training clips).
- Fit time: 5 min 02 s; 2,262 person detections.
- Checkpoint: `results/aivad/scene01/.../model.ckpt` (762.6 MB).

On the fight clip, raw scores were ~1.04 on a normal frame and 1.41-1.63 during the fight. Frame-level thresholds were noisy, so the renderer adds **temporal smoothing (window 15)**. A causal moving average plus threshold 1.3 and pixel threshold 1.1 produced a clean event segment; false positives clustered **outside** the fight (frames 23-35, 141-202, 398-403), so the demo was cropped to frames **260-420**.

Two practical issues surfaced:

1. **Empty-region bug:** AI-VAD crashes with `IndexError: amax(): Expected reduction dim 0 to have non-zero size` when no person is detected. The renderer catches it and treats the frame as normal.
2. **Causal smoothing tail:** the banner can lag the boxes; requiring at least one box fixed the mismatch.

Output:

- `results/videos/shanghaitech_scene01_01_0026_people_annotated.mp4`
- 161 frames (260-420), 10 fps, 856×480, 1.1 MB
- 91 frames with red person boxes; first/last boxed frame 311/405; max smoothed score 2.40
- Frames are labelled `PEOPLE ANOMALY` only when boxes are present

The renderer was corrected after inspection showed a walker boxed before the fight. The initial implementation ranked each detector box by the maximum anomaly-map value inside it. The final implementation uses AI-VAD's density-estimator per-region `box_scores`; frame 263 is no longer boxed, while the fight boxes remain.

This is a demonstration of the video-native branch of the project, not a benchmark result.

---

10. Hyperparameters used (first run)

PatchCore

Parameter	Value
`backbone`	`wide_resnet50_2`
`layers`	`["layer2", "layer3"]`
`pre_trained`	True (default)
`coreset_sampling_ratio`	0.01
`num_neighbors`	9
`precision`	float32
`image_size`	256×256
`train_batch_size`	16
`eval_batch_size`	16
`num_workers`	0
`clip_length_in_frames`	1
`frames_between_clips` (train)	100
`frames_between_clips` (test)	10
`max_epochs`	1
`visualizer`	False

Datamodule / evaluation

Parameter	Value
`target_frame`	last (default)
`val_split_mode`	same as test (default)
`test_split_mode`	from directory (default)
threshold	adaptive from validation
seed	not fixed yet

---

11. Challenges faced (complete list)

Every issue is documented in detail in `docs/CHALLENGES.md`. Summary:

#	Challenge	Outcome
1	User wanted video; MVTec-AD is images only	Pivoted to video benchmarks
2	Anomalib 2.7 outside Python 3.13 tested range	Created Python 3.10 venv
3	System PyTorch was CPU-only	Installed CUDA 12.8 wheels
4	OneDrive blocked automated ShanghaiTech download (403)	Used Hugging Face mirror

#	Challenge	Outcome
5	Chrome download disappeared; temp file sizes looked stale	Abandoned browser route
6	FDM could not download a OneDrive folder share	Explained folder vs direct link
7	torchvision 0.26 removed video decoding	Installed TorchCodec
8	TorchCodec could not find FFmpeg shared DLLs	Portable FFmpeg shared build
9	DLLs in `.venv/Scripts` were not found	Copied DLLs into `site-packages/torchcodec`
10	Avenue had an extra top-level folder	Moved dataset folders up
11	ShanghaiTech had no `training/frames`	Used `converted_videos` as Anomalib expects
12	Raw ShanghaiTech videos failed frame indexing	Let Anomalib re-encode to XVID
13	Image visualizer crashed on video batches	`visualizer=False`
14	PatchCore would need ~440 GB for 70k frames	Subsampled normal frames
15	Full-test pixel AUROC ran out of RAM (7.9 GB)	Subsample test frames / plan streaming metrics
16	Smoke test metrics were zero	Expected: limited test subset had no anomalies
17	DataLoader slow with `num_workers=0`	Known; can try 2–3 workers
18	Threshold warning: validation set has no anomalies	Threshold derived from normal scores; document it
19	HF cache symlink warning on Windows	Non-fatal; can enable Developer Mode
20	`triton not found` warning	Non-fatal on Windows; FLOP counting disabled
21	Many timm/deprecation warnings	Non-fatal; version drift in Anomalib
22	Dataset licenses restrict commercial use	Documented; research only

---

12. What still needs tuning

No systematic tuning has been done yet. The first run is a baseline with reasonable defaults. The following are planned and necessary.

12.1 Memory-bank size (most important)

- Current: every 100th normal frame (~700 frames).
- Alternatives: 400, 1,000, 2,000 frames.
- Trade-off: larger bank → better coverage → more RAM/time.
- Constraint: 7.9 GB system RAM.

12.2 Coreset ratio

- Current: 0.01.
- Alternatives: 0.001, 0.005, 0.01, 0.05, 0.1.
- Trade-off: smaller memory bank at test time vs AUROC and localization.

12.3 Backbone

- Current: `wide_resnet50_2`.
- Alternatives: `resnet50`, larger timm models, ViT/DINOv2.
- Trade-off: accuracy vs speed and memory.

12.4 Feature layers

- Current: `layer2 + layer3`.
- Alternatives: `layer2`, `layer3`, `layer1 + layer2 + layer3`.
- Trade-off: spatial resolution vs semantic depth.

12.5 Input resolution

- Current: 256.
- Alternatives: 224, 320.
- Resolution directly controls pixel-metric memory and inference time.

12.6 Neighbors

- Current: 9.
- Alternatives: 1, 5, 9.
- Affects score smoothing and speed.

12.7 Test sampling for evaluation

- Current: every 10th frame (1,204).
- Final benchmark should use every frame or a principled subset.
- Best solution: streaming/per-video AUROC that does not hold all pixel predictions.

12.8 PaDiM-specific

- `n_features`: 100 vs 550.
- covariance regularization (epsilon): 0.01 / 0.1 / 0.5.
- layers/backbone.

12.9 AI-VAD-specific

From the verified signature: `box_score_thresh`, `persons_only`, `min_bbox_area`, `max_bbox_overlap`, `enable_foreground_detections`, `foreground_kernel_size`, `foreground_binary_threshold`, `n_velocity_bins`, `use_velocity_features`, `use_pose_features`, `use_deep_features`, `n_components_velocity`, `n_neighbors_pose`, `n_neighbors_deep`.

Important because AI-VAD is the first true video-native model in the lineup.

12.10 FUVAS-specific

- `backbone`, `layer`, `spatial_pool`, `pooling_kernel_size`, `pca_level`.

- Few-shot `k`: 1, 2, 4, 8.

12.11 WinCLIP-specific

- `scales`: 1, 2, 3.
- Prompt templates (normal vs abnormal).
- `class_name` handling per scene.

12.12 Temporal layer

- Smoothing window: 1, 5, 11, 21 frames.
- Moving average vs median vs max.
- Hysteresis thresholds and debounce length.

12.13 Seeds and reproducibility

- Set seeds for Python, NumPy, PyTorch.
- Save the exact config with every run.
- Record package versions.

---

13. Current state

Ready

- Environment (GPU verified).
- All datasets.
- Data pipelines for Avenue and ShanghaiTech.
- One real PatchCore result.
- Documentation set.

Not ready

- `src/render_video.py` and `src/render_people_video.py` are implemented; the general runner, `configs/`, and `app/` remain pending.
- No PaDiM/FUVAS/WinCLIP benchmark runs yet; AI-VAD has a scene-1 fit and people-only demo.
- No temporal layer.
- No Gradio demo.
- No memory-safe full-test evaluation.
- No hyperparameter ablations.
- No report.

---

14. Next steps (ordered)

1. Write `src/run.py` — config-driven runner.
2. Implement per-scene memory-bank sampling (target ~700 frames, adjustable).
3. Implement memory-safe evaluation: - image AUROC on all test frames (cheap), - pixel AUROC on a fixed subsample or streaming per video.
4. Save and reuse checkpoints.
5. Run PaDiM scene 1 with the same protocol.
6. Run PaDiM + PatchCore on scenes 1–3.
7. Run AI-VAD on scenes 1–3 (scene 1 fit/demo is complete; scenes 2–3 remain).
8. Add temporal smoothing and event metrics.
9. Run FUVAS few-shot, then WinCLIP zero-shot.
10. Extend to all 13 scenes + Avenue.
11. Gradio demo + ONNX export + FPS table.
12. Final report and results table.

---

15. Lessons learned

1. **Check the dataset first.** "Latest industrial anomaly detection" and "video" are not compatible with the standard MVTec benchmark. Verify modality before planning.
2. **Check the Python support matrix.** Anomalib's metadata was permissive, but classifiers stopped at 3.12.
3. **Install CUDA torch first, then the framework, then re-verify.** Dependency graphs can silently replace GPU wheels.
4. **Video decoding is a dependency minefield.** torchvision 0.26 → TorchCodec → FFmpeg shared DLLs → correct DLL directory.
5. **Windows DLL search behavior matters.** App-directory copies did not work; the DLL's own directory did.
6. **Official large-dataset links can be unreliable.** A verified mirror plus byte-size checks saved the day.
7. **Frame counts are not free.** A 70k-frame scene is not a 70k-image dataset for memory-bank methods; it is two orders of magnitude larger.
8. **System RAM, not VRAM, was the bottleneck.** The GPU was never the problem; metric accumulation was.
9. **Document the exact protocol.** Subsampled training and evaluation must be stated or the numbers are misleading.
10. **Smoke-test metrics can legitimately be zero.** A limited test subset may contain no positives; that is not a pipeline failure.

---

16. Timeline (2026-11-02)

Time (approx)	Event
10:34	GPU audit (`nvidia-smi`)
11:00	Avenue + UCSD downloaded and extracted
12:26–12:44	ShanghaiTech browser download issues; temp file observed
12:44	Python 3.10 venv created; pip upgraded

Time (approx)	Event
12:44–13:02	CUDA torch, Anomalib, Gradio installed
13:02	GPU and model imports verified
13:42	Avenue datamodule failed: TorchCodec missing
13:44	FFmpeg shared build installed; DLL fix
13:46	ShanghaiTech datamodule verified (69,976 / 11,894)
13:50–14:14	ShanghaiTech re-encode to XVID (330 clips)
14:15	PatchCore smoke test (visualizer fix)
14:58	Real scene-1 run; full test OOM at AUROC
15:05	Re-test from checkpoint with stride 10; metrics obtained
15:15+	Documentation updated; journey authored

---

17. Appendix: key commands

Build environment

```
py -3.10 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu128
.\.venv\Scripts\python.exe -m pip install "anomalib==2.7.0" gradio
.\.venv\Scripts\python.exe -m pip install av "open-clip-torch" torchcodec
```

Verify GPU

```
.\.venv\Scripts\python.exe -c "import torch; print(torch.__version__, torch.cuda.is_available(),
torch.cuda.get_device_name(0))"
```

Join ShanghaiTech parts

```
cmd /c copy /b
shanghaitech.tar.gz.aa+shanghaitech.tar.gz.ab+shanghaitech.tar.gz.ac+shanghaitech.tar.gz.ad+shanghaitec
h.tar.gz.ae+shanghaitech.tar.gz.af+shanghaitech.tar.gz.ag shanghaitech.tar.gz
```

Extract

```
tar -xzf shanghaitech.tar.gz -C data\shanghaitech
```

Fix FFmpeg DLLs for TorchCodec

```
Copy-Item "<ffmpeg-bin>\*.dll" ".venv\Lib\site-packages\torchcodec\" -Force
```

Run a PatchCore experiment (conceptual)

```
from anomalib.data import ShanghaiTech
from anomalib.models import Patchcore
from anomalib.engine import Engine

dm_train = ShanghaiTech(root="data/shanghaitech", scene=1, clip_length_in_frames=1,
                        frames_between_clips=100, train_batch_size=16, num_workers=0)
dm_test = ShanghaiTech(root="data/shanghaitech", scene=1, clip_length_in_frames=1,
                       frames_between_clips=10, eval_batch_size=16, num_workers=0)

model = Patchcore(backbone="wide_resnet50_2", layers=["layer2","layer3"],
                  coreset_sampling_ratio=0.01, num_neighbors=9, visualizer=False)
engine = Engine(max_epochs=1, accelerator="gpu", devices=1,
                default_root_dir="results/patchcore/scene01")
engine.fit(model=model, datamodule=dm_train)
engine.test(model=model, datamodule=dm_test)
```

---

18. Appendix: file and artifact inventory

Documentation

File	Purpose
`README.md`	Entry point, status, first results
`PROJECT_GUIDE.md`	Reference manual, plan, runbooks
`docs/PROGRESS.md`	Running log
`docs/JOURNEY.md`	This complete journey
`docs/DECISIONS.md`	Decision records
`docs/CHALLENGES.md`	Problem catalog
`docs/EXPERIMENTS.md`	Experiment and tuning log
`docs/Project_Journey.pdf`	PDF build of the journey docs

Code and config

Path	State
`configs/`	Empty (planned)
`src/`	PatchCore and AI-VAD renderers; benchmark runner planned
`app/`	Empty (planned)

Path	State
`tests/`	Empty (planned)
`requirements.txt`	Written with verified versions
`results/summary.csv`	First result row

Data

Path	Size / contents
`data/avenue/`	~1.53 GB
`data/ucsdped/`	~1.77 GB
`data/shanghaitech/`	~24.4 GB (parts, joined archive, raw, converted)
`data/shanghaitech_parts/`	7 archive parts + joined archive

Models / results

Path	Contents
`results/patchcore/scene01/.../model.ckpt`	Fitted PatchCore checkpoint (~139.5 MB)
`results/smoke_patchcore/.../model.ckpt`	Smoke-test checkpoints

---

19. Appendix: glossary

Term	Meaning
AUROC	Area under the ROC curve; threshold-free ranking metric
Pixel AUROC	AUROC computed per pixel for localization
F1	Harmonic mean of precision and recall; needs a threshold
Coreset	A representative subsample of a larger set
Memory bank	Stored normal feature vectors used for nearest-neighbor scoring
Frame-wise	Treat each frame independently
Video-native	Use temporal information such as motion or clips
Few-shot	Learn from a small number of normal examples
Zero-shot	No target-dataset training
Temporal smoothing	Aggregate frame scores over time
TorchCodec	PyTorch video decoding library used by torchvision 0.26+
FFmpeg shared	FFmpeg build that ships DLLs, required by TorchCodec
Anomalib	Intel's anomaly-detection library
ShanghaiTech	13-scene campus video anomaly benchmark

Term	Meaning
Avenue	CUHK Avenue abnormal-event benchmark
UCSD Ped	UCSD pedestrian anomaly benchmark

Decision Log

Decision Log (ADR-style)

Every significant decision made during the project, with context, alternatives, the choice made, and its consequences. Status values: **Accepted**, **Superseded**, **Provisional**.

D-01 — Choose unsupervised anomaly detection as the project type

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** The user wanted a computer-vision project using current technology, not too hard, that demonstrates a strong ML/vision foundation.
- **Alternatives:** Detection + tracking + counting, text-prompted segmentation with Grounding DINO + SAM 2, monocular depth, CLIP retrieval, VLM fine-tuning.
- **Decision:** Unsupervised anomaly detection, because it teaches one-class learning, representation quality, localization, and rigorous evaluation — all common in industry.
- **Consequences:** Requires careful evaluation protocol (AUROC, pixel metrics, threshold rules). Strong interview signal. No need to collect defect labels.

D-02 — Use Anomalib instead of from-scratch implementations

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** The user chose a library-based route for speed and reproducibility.
- **Alternatives:** Implement PaDiM/PatchCore from scratch; hybrid; use Anomalib only as a cross-check.
- **Decision:** Anomalib 2.7.0.
- **Consequences:**
- Fast access to PaDiM, PatchCore, AI-VAD, FUVAS, WinCLIP.
- Built-in metrics, thresholding, checkpointing, Engine.
- Less "built from scratch" signal; compensated by experiments, ablations, and analysis.
- Must document library behavior and bugs (visualizer, video batches, TorchCodec).

D-03 — Dedicated Python 3.10 virtual environment

- **Date:** 2026-11-02
- **Status:** Accepted

- **Context:** System default is Python 3.13.9. Anomalib 2.7.0 declares `>=3.10` but its classifiers cover only 3.10–3.12.
- **Alternatives:** Use 3.13 and risk unsupported dependencies; install 3.12; use conda.
- **Decision:** Use the already-installed Python 3.10.11 in a local `.venv`.
- **Consequences:** Reproducible environment; isolated from system Python; slightly older interpreter than the machine default.

D-04 — Install CUDA PyTorch before Anomalib

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** System PyTorch was `2.10.0+cpu`. Anomalib depends on PyTorch and Lightning.
- **Alternatives:** Let Anomalib resolve PyTorch; install everything from PyPI.
- **Decision:** Install `torch` / `torchvision` from the CUDA 12.8 index first, then Anomalib, then re-verify CUDA.
- **Consequences:** GPU works (`2.11.0+cu128` , `cuda True`). Anomalib did not downgrade it, but the verification step remains mandatory for reproducibility.

D-05 — Pivot from industrial images to surveillance video

- **Date:** 2026-11-02
- **Status:** Accepted (supersedes the original MVTec-AD plan)
- **Context:** The user explicitly wanted video. Verified that MVTec-AD and MVTec AD 2 are still-image datasets with no video clips. Anomalib supports `Avenue` , `ShanghaiTech` , `UCSDped` and video-native models.
- **Alternatives:** Stay on MVTec images; synthesize video; use a custom industrial clip.
- **Decision:** Video anomaly detection on surveillance benchmarks. The user accepted surveillance as the domain.
- **Consequences:** Loses the "industrial" theme; gains real video benchmarks, temporal analysis, and video-native models. The original PaDiM/PatchCore/WinCLIP comparison is retained, with AI-VAD/FUVAS added.

D-06 — ShanghaiTech Campus as the primary dataset

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** Need a standard video anomaly benchmark with localization labels.
- **Alternatives:** Avenue (small, 47 events), UCSD Ped (small), XD-Violence / UCF-Crime (large, weakly labeled).
- **Decision:** ShanghaiTech Campus: 13 scenes, ~330 normal training clips, ~107 test clips, 130 abnormal events, frame and pixel masks.
- **Consequences:** More scenes and labels than Avenue; manual download required; heavier disk and preprocessing.

D-07 — Avenue and UCSD Ped as secondary datasets

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** Need a fast sanity-check dataset and a second benchmark.
- **Alternatives:** Only ShanghaiTech; add a large weakly supervised dataset.
- **Decision:** Avenue (small, 16/21 videos) and UCSD Ped1/Ped2 (frame folders).
- **Consequences:** Avenue enables quick iteration and cross-dataset comparison later. Adds storage and layout work but little risk.

D-08 — Start with 3 scenes, then all 13

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** Running all 13 scenes × 5 models with limited RAM would be slow.
- **Decision:** Scenes 1–3 for iteration; all 13 for the final benchmark.
- **Consequences:** Faster feedback; final table requires a longer run later.

D-09 — Model lineup: PaDiM, PatchCore, AI-VAD, FUVAS, WinCLIP

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** Need a progression story from classical to modern to foundation models.
- **Alternatives:** Only PaDiM + PatchCore + WinCLIP (original image plan); add EfficientAd.
- **Decision:** PaDiM (classical), PatchCore (modern frame-wise), AI-VAD (video-native), FUVAS (few-shot video), WinCLIP (zero-shot).
- **Consequences:** Covers frame-wise vs video-native and trained vs few-shot vs zero-shot. More models to run; AI-VAD/FUVAS may need extra weights and tuning.

D-10 — Add WinCLIP last

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** Zero-shot CLIP adds a separate dependency and prompt engineering.
- **Decision:** Finish baselines and the eval harness first; add WinCLIP after.
- **Consequences:** Lower risk of blocking progress on CLIP setup.

D-11 — Gradio for the demo

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** Need an upload-and-see-result interface for videos.
- **Alternatives:** Streamlit, CLI only.
- **Decision:** Gradio.
- **Consequences:** Fast video upload and annotated output; easy local hosting.

D-12 — Frame-wise image models on video

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** PaDiM/PatchCore expect images. Anomalib's video datamodule can yield clips.
- **Alternatives:** Skip image models; adapt models to clips.
- **Decision:** Use `clip_length_in_frames=1` for PaDiM/PatchCore/WinCLIP, producing normal `(B, 3, H, W)` batches; use clips only for AI-VAD/FUVAS.
- **Consequences:** Image models run unmodified; they remain blind to motion, which is analyzed explicitly in the report.

D-13 — Disable the image visualizer for video data

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** Anomalib's `ImageVisualizer` crashed: `'VideoItem' object has no attribute 'image_path'`.
- **Alternatives:** Patch the visualizer; suppress the callback; use the video visualizer.
- **Decision:** Pass `visualizer=False` when using image models on video data.
- **Consequences:** Tests complete and metrics are correct; visual heatmap rendering must be produced by our own plotting code.

D-14 — Subsample normal frames for the PatchCore memory bank

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** 69,976 frames × 1,024 patches × 1,536 dims × 4 bytes ≈ 440 GB for scene 1. System RAM is 7.9 GB.
- **Alternatives:** Full frames (impossible); lower resolution (helps but insufficient); feature dimension reduction; random subset.
- **Decision:** Use every 100th normal frame (~700 frames) for the first run. Make this configurable per scene.
- **Consequences:** Feasible memory and time. The memory bank may miss rare normal patterns; this must be reported and ablated (400 / 700 / 1,000 / 2,000 frames).

D-15 — Subsample test frames for pixel metrics

- **Date:** 2026-11-02
- **Status:** Provisional
- **Context:** Full test (11,894 frames) ran inference but AUROC tried to allocate 6.2 GB and crashed on a 7.9 GB machine.
- **Alternatives:** 1. Subsample test frames (current). 2. Lower input resolution. 3. Compute image AUROC on all frames and pixel AUROC on a subset. 4. Stream per-video metrics without accumulating all pixel predictions.
- **Decision:** Use `frames_between_clips=10` (1,204 frames) for the first real number. Long-term, implement streaming/per-video evaluation.
- **Consequences:** Metrics are valid for the sampled subset but not the full benchmark. The report must state the protocol. Final results must improve this.

D-16 — Use the Hugging Face mirror for ShanghaiTech

- **Date:** 2026-11-02
- **Status:** Accepted (Superseded the OneDrive route)
- **Context:** Official OneDrive folder share returned 403 to automated requests; Chrome download disappeared; FDM could not enumerate a folder share.
- **Alternatives:** Manual browser download; JDownloader 2; other mirrors.
- **Decision:** Download the 7-part archive from `123zdm/ShanghaiTech` on Hugging Face.
- **Consequences:** Reliable and fast. Integrity verified by expected byte sizes and a byte-exact join. The mirror is third-party, so cite the original dataset and license.

D-17 — Accept Anomalib's XVID re-encode for ShanghaiTech

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** Hardlinking raw videos caused `Requested next frame while there are no more frames left to decode`.
- **Alternatives:** Skip bad videos; patch the indexer; decode with OpenCV in a custom dataset.
- **Decision:** Remove hardlinks and let Anomalib re-encode all 330 training videos.
- **Consequences:** 330 extra files (~2.28 GB) and 15–25 minutes of one-time work, but stable frame indexing and training.

D-18 — Store FFmpeg shared DLLs inside the TorchCodec package

- **Date:** 2026-11-02
- **Status:** Accepted

- **Context:** TorchCodec could not load FFmpeg. Adding to `PATH` worked only for that process; copying to `.venv/Scripts` did not work.
- **Alternatives:** Modify user/system PATH; copy DLLs into every working directory; use a launcher script.
- **Decision:** Copy the FFmpeg DLLs into `.venv/Lib/site-packages/torchcodec/`.
- **Consequences:** Works without environment changes. The fix is local to the venv and must be repeated if the venv is rebuilt.

D-19 — Keep both the split parts and the joined archive (for now)

- **Date:** 2026-11-02
- **Status:** Provisional
- **Context:** Data occupies ~24.4 GB; the 7 parts and the joined tar.gz duplicate data.
- **Alternatives:** Delete parts after joining; delete joined archive after extraction.
- **Decision:** Keep them until the final extraction is confirmed and a report is written.
- **Consequences:** Uses ~13 GB extra. Can be cleaned later to free space.

D-20 — Document the full journey in Markdown and PDF

- **Date:** 2026-11-02
- **Status:** Accepted
- **Context:** The user asked for a complete record of decisions, challenges, tuning, and the project journey, in MD and PDF.
- **Alternatives:** Only README; only a PDF; only chat history.
- **Decision:** Create `docs/JOURNEY.md`, `docs/DECISIONS.md`, `docs/CHALLENGES.md`, `docs/EXPERIMENTS.md`, and build `docs/Project_Journey.pdf` from them using Markdown → HTML (Python `markdown`) → Chrome headless print-to-PDF.
- **Consequences:** Reproducible documentation pipeline; the PDF can be regenerated after future experiments.

Superseded / changed decisions

Original	Replaced by	Reason
Industrial image anomaly detection on MVTec-AD	Video anomaly detection on ShanghaiTech/Avenue/UCSD	User wanted video; MVTec has no video
15 MVTec categories	13 ShanghaiTech scenes	Dataset pivot
PatchCore + WinCLIP only	PaDiM, PatchCore, AI-VAD, FUVAS, WinCLIP	Video-native and few-shot methods added
OneDrive browser download	Hugging Face mirror	403 / unreliable download
Python 3.13	Python 3.10 venv	Anomalib tested range

Original	Replaced by	Reason
CPU PyTorch	CUDA 12.8 PyTorch	GPU acceleration

Challenge Catalog

Challenge Catalog

Every notable problem encountered during the project, with the exact symptom, the root cause, the fix applied, the current status, and the lesson. Ordered roughly by when it appeared.

C-01 — "Industrial anomaly detection" has no standard video benchmark

- **Symptom:** The user asked for video, but the planned datasets were MVTec-AD / MVTec AD 2.
- **Root cause:** MVTec-AD and MVTec AD 2 are still-image datasets. They contain no video clips.
- **Evidence:** Official dataset pages; Anomalib video datamodules are only `Avenue`, `ShanghaiTech`, `UCSDped`.
- **Fix:** Pivoted to surveillance video anomaly detection; user accepted the domain change.
- **Status:** Resolved.
- **Lesson:** Verify the data modality before committing to a dataset/method plan.

C-02 — Anomalib 2.7.0 outside the system Python's tested range

- **Symptom:** System Python is 3.13.9. Anomalib 2.7.0 classifiers stop at 3.12.
- **Root cause:** Metadata says `requires_python >=3.10`, but published classifiers only cover 3.10, 3.11, 3.12.
- **Fix:** Created `.venv` with the already-installed Python 3.10.11.
- **Status:** Resolved.
- **Lesson:** Read classifiers, not only `requires_python`.

C-03 — System PyTorch was CPU-only

- **Symptom:** `torch.__version__` = `2.10.0+cpu`; `torch.cuda.is_available()` = False.
- **Root cause:** The default Python environment had a CPU wheel.
- **Fix:** Installed torch/torchvision from the CUDA 12.8 index first; verified CUDA True.
- **Status:** Resolved.
- **Lesson:** Install the GPU wheel before frameworks that depend on PyTorch.

C-04 — The official ShanghaiTech OneDrive link blocked automated downloads

- **Symptom:**
- `api.onedrive.com/.../shares/u!...` returned HTTP 400.
- Short link returned 301 to `onedrive.live.com`, then 403 Forbidden.
- **Root cause:** The share is a web folder and/or requires a browser session; the API forms tried were not accepted.
- **Fix:** Used a Hugging Face mirror of the same 7-part archive.
- **Status:** Resolved via mirror.
- **Lesson:** Have a verified mirror plan for large public datasets.

C-05 — Chrome download vanished, and file sizes looked wrong

- **Symptom:**
- `splits.zip` download reached at least ~904 MB.
- `data\Unconfirmed 267844.crdownload` later disappeared; no `splits.zip` existed.
- `Get-ChildItem` reported 11.1 MB while `[System.IO.FileInfo].Length` reported hundreds of MB for the same file.
- **Root cause:** Chrome held the file open and filesystem metadata was stale; the download was ultimately canceled/discarded.
- **Fix:** Abandoned the browser route; used the HF mirror. For diagnostics, read `[System.IO.FileInfo]::new(path).Length` instead of cached directory listings.
- **Status:** Resolved.
- **Lesson:** Do not trust a single size reading for an in-progress browser download.

C-06 — Download manager could not handle the OneDrive folder

- **Symptom:** FDM: "Server has returned a web page. Click to open it in the web browser."
- **Root cause:** The pasted URL is a folder share page, not a direct file URL. Folder shares cannot be enumerated by a simple download manager.
- **Fix:** Explained direct-link vs folder-link; recommended JDownloader 2 for folders, or the HF mirror (chosen).
- **Status:** Resolved.
- **Lesson:** Download managers need direct file URLs; folder shares need a browser or a specialized mirror.

C-07 — torchvision 0.26 removed video decoding

- **Symptom:** `ImportError: Video decoding capabilities were removed from torchvision and migrated to TorchCodec.`
- **Root cause:** torchvision 0.26 delegates video decoding to TorchCodec.
- **Fix:** `pip install torchcodec`.
- **Status:** Resolved (but see C-08).
- **Lesson:** Major version upgrades can remove long-standing functionality.

C-08 — TorchCodec could not find FFmpeg shared libraries

- **Symptom:** `RuntimeError: Could not load libtorchcodec. Likely causes: 1. FFmpeg is not properly installed... On Windows, ensure you've installed the "full-shared" version which ships DLLs.`
- **Root cause:** No FFmpeg was installed; TorchCodec needs shared `avcodec-*.dll`, `avformat-*.dll`, etc. A static `ffmpeg.exe` is not sufficient.
- **Fix:** Downloaded the BtbN portable shared build (90,941,653 bytes), FFmpeg 9 / `avcodec-63.dll`.
- **Status:** Resolved (with C-09).
- **Lesson:** "FFmpeg installed" can mean a static binary; shared libraries are different.

C-09 — DLL directory mattered: `.venv/Scripts` did not work

- **Symptom:** After copying FFmpeg DLLs into `.venv\Scripts`, TorchCodec still failed with `Could not find module ... or one of its dependencies`.
- **Root cause:** Windows resolves dependent DLLs relative to the loaded DLL's directory, not reliably from the application directory in this setup.
- **Fix:** Copied the DLLs into `.venv\Lib\site-packages\torchcodec\` (next to `libtorchcodec_core9.dll`).
- **Status:** Resolved.
- **Lesson:** Put dependent DLLs next to the extension DLL that needs them.

C-10 — Avenue extracted with an extra top-level folder

- **Symptom:** Anomalib's `Avenue` datamodule expects `training_videos/` and `testing_videos/` directly under the root, but the zip created `Avenue Dataset/`.
- **Root cause:** Archive layout differs from Anomalib's assumption.
- **Fix:** Moved `training_videos`, `testing_videos`, `training_vol`, `testing_vol` up one level; left `ground_truth_demo/` at the root.
- **Status:** Resolved.
- **Lesson:** Normalize dataset layout to the library's expected structure before debugging the model.

C-11 — ShanghaiTech had no `training/frames/`

- **Symptom:** Documentation implied frame folders, but the archive only contained `training/videos/*.avi`.
- **Root cause:** Dataset distribution difference.
- **Fix:** Confirmed Anomalib training uses `training/converted_videos/*.avi`; testing uses `testing/frames/<clip>/*.jpg`.
- **Status:** Resolved.
- **Lesson:** Read the dataset class source, not just the docs.

C-12 — Raw ShanghaiTech videos failed frame indexing

- **Symptom:** `RuntimeError: Requested next frame while there are no more frames left to decode.`
- **Root cause:** Raw ShanghaiTech videos have encoding quirks. Hardlinking them into `converted_videos/` skipped Anomalib's intended re-encode and exposed the issue.
- **Fix:** Removed the hardlinks; ran `prepare_data()`, which re-encoded all 330 training videos to XVID.
- **Status:** Resolved.
- **Lesson:** Respect the library's data preparation for known-problematic encodings.

C-13 — Image visualizer crashed on video batches

- **Symptom:** `AttributeError: 'VideoItem' object has no attribute 'image_path'`
- **Root cause:** Anomalib's `ImageVisualizer` assumes image datasets; video batches carry `video_path`.
- **Fix:** Pass `visualizer=False` to image models used on video data.
- **Status:** Resolved (workaround).
- **Lesson:** Metrics and visualization are separate concerns; disable the incompatible one.

C-14 — PatchCore memory explosion with 70k frames

- **Symptom:** Estimated ~440 GB of patch features for scene 1.
- **Root cause:** PatchCore stores every patch embedding; 69,976 frames × 1,024 patches × 1,536 dims × 4 bytes.
- **Fix:** Build the memory bank from every 100th normal frame (~700 frames).
- **Status:** Mitigated; needs principled ablation.
- **Lesson:** Frame-wise methods scale with frames, not videos; sample deliberately.

C-15 — Full-test pixel AUROC ran out of RAM

- **Symptom:** `DefaultCPUAllocator: not enough memory: you tried to allocate 6,235,881,472 bytes` after 33 min of successful inference on 11,894 frames.
- **Root cause:** TorchMetrics accumulated ~780 million pixel predictions and sorted them; the machine has 7.9 GB RAM.
- **Fix (temporary):** Re-tested from checkpoint with every 10th frame (1,204 frames).
- **Status:** Open engineering issue for the full benchmark.
- **Lesson:** The bottleneck is metric accumulation and CPU RAM, not GPU or inference.

C-16 — Smoke-test metrics were all zero

- **Symptom:** `image_AUROC = 0`, `pixel_AUROC = 0` after a limited test.
- **Root cause:** `No positive samples in targets, true positive value should be meaningless.` The 2-batch test subset contained only normal frames.
- **Fix:** None needed; this was expected for the smoke test.
- **Status:** Understood.
- **Lesson:** A limited test subset can legitimately yield zero metrics.

C-17 — Validation split contained no anomalies

- **Symptom:** "The validation set does not contain any anomalous images. As a result, the adaptive threshold will take the value of the highest anomaly score observed in the normal validation images..."
- **Root cause:** Default validation split draws from test; with a small/limited split it may contain only normal frames.
- **Fix:** Accept for AUROC (threshold-free); for F1, document that the threshold is derived from normal scores. Improve the validation protocol later.
- **Status:** Open protocol decision.
- **Lesson:** Threshold selection needs an explicit, anomaly-containing validation split or a different rule.

C-18 — Data loading is slow with `num_workers=0`

- **Symptom:** Test inference ~5.9 fps; Lightning warns about missing workers.
- **Root cause:** Video decoding and JPEG loading run in the main process.
- **Fix (proposed):** Try `num_workers=2–3`; measure stability and RAM.
- **Status:** Open optimisation.
- **Lesson:** Windows dataloading needs care; workers can help but cost RAM.

C-19 — Hugging Face cache symlink warning on Windows

- **Symptom:** "huggingface_hub cache-system uses symlinks by default ... your machine does not support them."
- **Root cause:** Windows symlink support requires Developer Mode or admin.
- **Fix:** None needed; cache still works with extra disk usage.
- **Status:** Non-blocking.
- **Lesson:** Warnings are not always errors.

C-20 — `triton not found` warning

- **Symptom:** "triton not found; flop counting will not work for triton kernels."
- **Root cause:** Triton is not available on this Windows setup.

- **Fix:** None needed; FLOP counting is optional.
- **Status:** Non-blocking.
- **Lesson:** Ignore optional tooling warnings.

C-21 — Deprecation warnings from timm/Anomalib

- **Symptom:** `timm.models.layers` deprecation; `anomalib.models.components.dinov2` deprecation.
- **Root cause:** Version drift inside Anomalib.
- **Fix:** None needed now; pin versions.
- **Status:** Non-blocking.
- **Lesson:** Pin versions for reproducibility.

C-22 — Dataset licenses restrict commercial use

- **Symptom:** ShanghaiTech is BSD-2-Clause; Avenue is academic-only; UCSD is research-use.
- **Root cause:** Standard academic dataset licensing.
- **Fix:** Documented in README and guide; project treated as research/portfolio.
- **Status:** Documented.
- **Lesson:** Always state dataset licenses.

C-23 — `Get-ChildItem` showed stale sizes for an open file

- **Symptom:** The same `.crdownload` appeared as 11.1 MB and ~837 MB in different commands.
- **Root cause:** Directory metadata caching while Chrome held the file open.
- **Fix:** Used `[System.IO.FileInfo]::new(path).Length` and a 4-second delta measurement.
- **Status:** Resolved.
- **Lesson:** For live files, query the file object directly and sample twice.

C-24 — Re-encode output directories were missing

- **Symptom:** `training/converted_videos` and `training/frames` did not exist in the archive.
- **Root cause:** The distribution only ships `training/videos`.
- **Fix:** Created `training/frames` and `training/converted_videos` directories; let `prepare_data()` populate converted videos.
- **Status:** Resolved.
- **Lesson:** Create expected directories before assuming a library will.

C-25 — Training "instant start" after download was not realistic

- **Symptom:** Expectation: start training immediately when the archive finished.
- **Root cause:** Unavoidable steps: join/extract 6.5 GB, re-encode 330 videos, build the venv, install 2.5 GB of torch, configure TorchCodec/FFmpeg, and produce a memory bank.
- **Fix:** Parallelized environment setup with the download, and prepared a watcher-style runbook. Communicated the real prep window.
- **Status:** Understood.
- **Lesson:** "Instant" is a sequence of prerequisites; list them explicitly.

Open technical issues (summary)

Issue	Priority	Next action
Memory-safe full-test evaluation	High	Stream metrics per video or use two-tier evaluation
Memory-bank size ablation	High	400 / 700 / 1,000 / 2,000 frames
Validation threshold protocol	High	Build a validation split with anomalies or define a rule
Data loading speed	Medium	Try `num_workers=2–3`
Reproducible seeds	Medium	Seed Python/NumPy/PyTorch, log configs
Visual heatmaps	Medium	Generate overlays ourselves (visualizer disabled)
FUVAS/WinCLIP resource use	Medium	Test at low resolution first
Disk cleanup	Low	Remove duplicate split parts after final verification

Experiments, Hyperparameters, and Tuning

Experiment Log, Hyperparameters, and Tuning Plan

All experiments run on 2026-11-02 unless stated otherwise.

Environment: Python 3.10.11, torch 2.11.0+cu128, torchvision 0.26.0+cu128, Anomalib 2.7.0, RTX 3060 12 GB, 7.9 GB system RAM.

---

1. Evaluation protocol (current)

Rules

- Fit on normal training data only.
- Evaluate on the test split.
- Report frame/image AUROC as the threshold-free main metric.
- Report pixel AUROC and pixel F1 for localization.
- Threshold for F1 comes from Anomalib's adaptive threshold on the validation split.
- Record the exact sampling protocol; never compare numbers across different protocols.

Current memory-safe compromise

Because of the 7.9 GB RAM limit:

- **Training memory bank:** every `N` th normal frame (configurable; first run N=100).
- **Test set:** every `M` th frame for pixel metrics (first real run M=10).
- Image-level AUROC on all test frames is cheap; pixel AUROC on all frames is not.

The final benchmark should replace M with a streaming/per-video metric so that all test frames are evaluated without materialising all pixel predictions at once.

---

2. Experiment E-000 — Smoke tests

E-000a — Avenue data pipeline

Command (conceptual):

```
dm = Avenue(root="data/avenue", gt_dir="data/avenue/ground_truth_demo",
            clip_length_in_frames=1, batch_size=2, num_workers=0)
dm.prepare_data(); dm.setup()
```

Result:

- train samples: 15,328
- test samples: 15,324
- image batch: `(2, 3, 480, 856)`

Purpose: confirm the datamodule and preprocessing work.

E-000b — PatchCore limited fit/test

Configuration:

- scene 1, clip length 1
- batch size 8
- `limit_train_batches=5`, `limit_val_batches=2`, `limit_test_batches=2`
- PatchCore, WRN50, layers 2+3, coreset 0.01, neighbors 9
- `visualizer=False`

Result:

- Fit: success, GPU used, coreset selected over 408 candidates.
- Test: success after disabling the visualizer.
- Metrics: 0.0 because the limited test subset had no anomalous frames.

Purpose: verify GPU training/inference and identify the visualizer bug.

E-000c — Full-test failure

Configuration:

- Same fitted model.
- Test datamodule stride 1 (11,894 frames).

Result:

- Inference completed (744 batches, 33 min 52 s).
- AUROC computation failed allocating 6.2 GB.

Purpose: establish the RAM ceiling; not a valid result.

---

3. Experiment E-001 — PatchCore, ShanghaiTech scene 1

Status

Completed (subsampled protocol).

Configuration

Item	Value
Model	PatchCore

Item	Value
Backbone	`wide_resnet50_2`
Layers	`layer2`, `layer3`
Pretrained	yes
Input size	256×256
Coreset ratio	0.01
Neighbors	9
Precision	float32
Train sampling	every 100th normal frame
Train clips	~700 (47 batches × 16)
Test sampling	every 10th test frame
Test clips	1,204
Train batch size	16
Eval batch size	16
Workers	0
Max epochs	1
Visualizer	disabled
Threshold	adaptive from validation
Seed	not fixed

Timings

Stage	Time
Fit (including feature extraction + coreset)	239.5 s
Test inference + metrics (1,204 frames)	379.9 s
Full-test inference before OOM (11,894 frames)	2,032 s

Results

Metric	Value
image_AUROC	0.78525
image_F1Score	0.67164
pixel_AUROC	0.98871
pixel_F1Score	0.59476

Artifacts

```
results/patchcore/scene01/Patchcore/ShanghaiTech/v0/weights/lightning/model.ckpt
```

Observations

- Pixel localization is much stronger than frame-level detection.
- A frame-wise model likely misses motion-only anomalies (running, chasing).
- The memory bank is small (~700 frames), which may limit coverage of normal variation.
- Test sampling changes the metric; these numbers are not directly comparable to published full-test results.

What this experiment does **not** establish

- Full-test performance.
- Cross-scene generalization.
- A fair head-to-head against video-native methods.
- The effect of hyperparameters.

3.5 Experiment E-002 — Annotated demo video (visualization)

Status

Completed.

Goal

Produce a watchable MP4 with red boxes around detected anomalies, using the E-001 PatchCore checkpoint.

Source clip

- ShanghaiTech scene 1, clip `01_0136`
- 529 frames, 856×480
- Ground-truth frame mask: 311 anomalous frames (first positive index 133)

Renderer

`src/render_video.py`, loading:

```
results/patchcore/scene01/Patchcore/ShanghaiTech/v0/weights/lightning/model.ckpt
```

Pipeline per frame:

1. Read JPEG (BGR) → RGB → float/255.
2. Apply the checkpoint's pre-processor: resize 256×256 + ImageNet normalize.
3. Forward pass through the PatchCore torch model.
4. Threshold the 256×256 anomaly map, morphological open/close.
5. Upscale the mask to 856×480, find contours, filter by area, keep top-k.

6. Draw red rectangles; draw a banner with `ANOMALY` / `NORMAL`, raw score, box count.
7. Write at 10 fps.

Display settings

Setting	Value	Note
Pixel threshold	55.0	checkpoint value is 48.84
Image threshold	58.0	checkpoint value is 46.27
Min box area	200 px	at 856×480
Top-k boxes	3	per frame
Boxes drawn only if frame is anomalous	yes	prevents boxes on NORMAL frames
FPS	10	ShanghaiTech test frames have no fps metadata

Why the display thresholds differ from the checkpoint

The checkpoint thresholds produced many small boxes on normal frames (for example frames 62 and 80 on a backpack). A threshold sweep on this clip gave:

Image threshold	Precision	Recall	F1
46.3 (checkpoint)	0.66	1.00	0.80
50	0.79	0.97	0.87
52	0.89	0.93	0.91
54	0.97	0.89	0.93
55	0.97	0.87	0.92
58	0.99	0.79	0.88
60	0.99	0.70	0.82

Pixel threshold 55 removed the small false boxes while keeping the truck box.

At image threshold 58 the two remaining "false positives" are frames 271 and 323. Visual inspection shows the **truck is clearly present** in both, while the frame-level GT mask is 0. This is likely sparse annotation rather than a model error, and is recorded as an error-analysis observation.

Results

Item	Value
Frames rendered	529
Render time	19.8 s (~26.7 fps)
Frames labelled ANOMALY	247
Frames with boxes	245
First / last boxed frame	169 / 431

Item	Value
Max raw score	82.35 (truck)
Video size	4.4 MB
Resolution / fps	856×480 @ 10 fps

Artifacts

```
results/videos/shanghaitech_scene01_01_0136_annotated.mp4
results/videos/shanghaitech_scene01_01_0136_annotated.scores.csv
results/videos/01_0136_samples/frame_007.jpg   (NORMAL, no box)
results/videos/01_0136_samples/frame_169.jpg   (first boxed frame)
results/videos/01_0136_samples/frame_297.jpg   (peak truck box, score 82.3)
results/videos/01_0136_samples/frame_309.jpg   (high-confidence box)
```

Caveats

- Display thresholds are tuned for presentation on this clip; the reported metrics use the checkpoint thresholds and the E-001 protocol.
- Frame-wise detection still misses low-score early/late anomaly frames.
- The demo intentionally focuses on the clear truck event.

3.6 Experiment E-003 — AI-VAD people-only demo video

Why this experiment exists

The frame-wise PatchCore model was tested on the fight clip `01_0026` (two people brawling, GT span 278-397). Scores stayed at 44-56 and produced **no boxes**. A scan of all 34 scene-1 clips showed every high-scoring clip was a vehicle (van, car, three-wheeler). The model is blind to motion anomalies. A video-native model is required.

Model and fit

Item	Value
Model	`AiVad`
`persons_only`	True
`enable_foreground_detections`	False
`pre_processor` / `post_processor`	False
Data	ShanghaiTech scene 1, `clip_length_in_frames=2`, `frames_between_clips=100`
Train clips	738
Batch size	2
Workers	0

Item	Value
Max epochs	1 (density estimation, no gradient training)
Validation	disabled (`limit_val_batches=0`)
Components	Mask R-CNN, Keypoint R-CNN, RAFT optical flow, CLIP ViT-B/16, velocity/pose/deep density estimators
Fit time	**5 min 02 s**
Person detections	**2,262**
Checkpoint	`results/aivad/scene01/AiVad/ShanghaiTech/v0/weights/lightning/model.ckpt` (762.6 MB)

Clip and calibration

- Clip: `01_0026`, 505 frames, 856×480.
- GT frame mask: 120 positive frames, span 278-397.
- A score-only pass over 504 frame pairs took **3.7 min**.
- Raw image scores: normal frames around **1.04**, fight frames **1.41-1.63**, max 2.45.
- Frame-level separation is noisy because normal crowd motion also produces people regions.
- Temporal smoothing (window 15) improves frame-level F1 to **0.73** at threshold 1.2.
- Remaining false positives are concentrated **outside** the fight: frames 23-35, 141-202, and 398-403.
- Display choices: **smooth window 15**, **image threshold 1.3**, **box threshold 1.1**, **min area 200**, **top-k 3**, cropped to frames **260-420**.

Render result

Item	Value
Processed pairs	176 (frames 245-420 for warmup)
Output frames	161 (frames 260-420)
Render time	169.3 s (~1.0 fps)
Frames labelled ANOMALY	101
Frames with boxes	91
First / last boxed frame	311 / 405
Max smoothed score	2.40
Output	856×480 @ 10 fps, 1.1 MB

Artifacts

```
results/videos/shanghaitech_scene01_01_0026_people_annotated.mp4
results/videos/shanghaitech_scene01_01_0026_people_annotated.scores.csv
results/videos/01_0026_people_samples/
```

Notes and bugs

- **Empty-region bug:** AI-VAD raises `IndexError: amax(): Expected reduction dim 0 to have non-zero size` on frames with no person detections. The renderer catches this and treats the frame as normal (score 1.0, empty map).
- **Causal smoothing:** the moving average lags and tails, so the banner requires at least one box to say `PEOPLE ANOMALY`.
- **Region-score correction:** the first renderer ranked each person by the maximum anomaly-map value inside its box. The corrected renderer now uses the density estimator's per-region `box_scores`, which removes the earlier walker box at frame 263 while retaining the fight event.
- **Display thresholds** are tuned for presentation, not a benchmark protocol.
- **GT sparsity / crowd motion** produce false positives outside the fight segment, which is why the demo is cropped to the event.
- This is a visualization result. It does not change the metric protocol.

---

4. Current hyperparameter table

PatchCore

Hyperparameter	Current	Typical range	Notes
backbone	wide_resnet50_2	resnet50, WRN50, ViT	accuracy vs speed
layers	layer2+3	layer2, layer3, layer2+3	resolution vs semantics
coreset_sampling_ratio	0.01	0.001–0.1	bank size vs accuracy
num_neighbors	9	1–9	score smoothing
precision	float32	float16, float32	speed vs exactness
image_size	256	224–320	memory and detail
memory-bank frames	~700	400–2,000	biggest accuracy lever
test sampling	every 10th	1–10	metric memory

PaDiM (planned)

Hyperparameter	Planned initial	Alternatives
backbone	wide_resnet50_2	resnet50, ViT
layers	layer2+3	layer2, layer3
n_features	100	550
covariance regularization	default in Anomalib	0.01, 0.1, 0.5
image_size	256	224, 320
memory-bank frames	~700	400–2,000

AI-VAD (planned)

Constructor parameters verified in Anomalib 2.7:

Parameter	Purpose
`box_score_thresh`	person-detector confidence threshold
`persons_only`	restrict to person regions
`min_bbox_area`	ignore tiny boxes
`max_bbox_overlap`	merge overlapping boxes
`enable_foreground_detections`	foreground segmentation
`foreground_kernel_size`	morphology kernel
`foreground_binary_threshold`	foreground threshold
`n_velocity_bins`	motion histogram bins
`use_velocity_features`	motion features on/off
`use_pose_features`	pose features on/off
`use_deep_features`	deep appearance features on/off
`n_components_velocity`	PCA components for velocity
`n_neighbors_pose`	kNN for pose
`n_neighbors_deep`	kNN for deep features

Initial plan: defaults, clip length ≥2, then tune `box_score_thresh`, `persons_only`, `min_bbox_area`, and the feature toggles.

FUVAS (planned)

Parameter	Purpose
`backbone`	feature extractor
`layer`	which layer
`spatial_pool`	pooling strategy
`pooling_kernel_size`	pooling size
`pca_level`	PCA reduction
few-shot `k`	1, 2, 4, 8

WinCLIP (planned)

Parameter	Purpose
`class_name`	object class for prompts
`k_shot`	0 for zero-shot; >0 for few-shot
`scales`	window scales
`few_shot_source`	reference images for few-shot

Initial plan: zero-shot, scales 2, then prompt/scale ablation.

---

5. Tuning plan (priority order)

T1 — Memory-bank size (highest impact)

- Values: 400, 700, 1,000, 2,000 normal frames per scene.
- Fixed: model/backbone/coreset/test protocol.
- Measure: image AUROC, pixel AUROC, fit time, peak RAM.
- Goal: find the knee where more frames stop helping.

T2 — Coreset ratio

- Values: 0.001, 0.005, 0.01, 0.05, 0.1.
- Fixed: memory-bank size.
- Measure: AUROC vs memory bank size and test FPS.
- Goal: maximise accuracy per stored vector.

T3 — Backbone

- Values: `resnet50`, `wide_resnet50_2`, a ViT/DINOv2 feature extractor.
- Fixed: best memory-bank size and coreset.
- Measure: AUROC, fit/test time, VRAM.
- Goal: test whether stronger features compensate for less data.

T4 — Feature layers

- Values: layer2, layer3, layer2+3.
- Measure: localization (pixel AUROC) vs semantic detection (image AUROC).

T5 — Input resolution

- Values: 224, 256, 320.
- Measure: AUROC vs inference speed and pixel-metric RAM.
- Note: 320 may exceed the safe RAM budget for full pixel metrics.

T6 — Neighbors

- Values: 1, 5, 9.
- Measure: AUROC and runtime.

T7 — Test-evaluation protocol

- Option A: every frame for image AUROC; every 10th for pixel AUROC.
- Option B: streaming per-video AUROC for both.

- Goal: remove sampling bias from the final numbers.

T8 — PaDiM-specific

- `n_features`: 100 vs 550.
- Covariance regularization: 0.01 / 0.1 / 0.5.
- Compare against PatchCore at matched memory-bank size.

T9 — AI-VAD

- Enable/disable velocity, pose, and deep features.
- Tune `box_score_thresh`, `min_bbox_area`, `persons_only`.
- Compare with frame-wise models on motion-heavy scenes.

T10 — FUVAS

- Few-shot curve: k = 1, 2, 4, 8.
- Backbone/layer/PCA level.

T11 — WinCLIP

- Prompt templates (normal vs abnormal).
- Scales 1 / 2 / 3.
- Zero-shot vs few-shot.

T12 — Temporal layer

- Smoothing window: 1, 5, 11, 21.
- Moving average vs median vs max.
- Hysteresis and debounce.
- Measure false alarms per minute and detection delay.

---

6. Planned experiment matrix

ID	Model	Data	Scene(s)	Purpose	Status
E-001	PatchCore	ShanghaiTech	1	first real baseline	Done
E-002	PaDiM	ShanghaiTech	1	classical baseline	Planned
E-003	PatchCore	ShanghaiTech	2, 3	scene generalization	Planned
E-004	PaDiM	ShanghaiTech	2, 3	scene generalization	Planned
E-005	PatchCore	ShanghaiTech	1	memory-bank size ablation	Planned
E-006	PatchCore	ShanghaiTech	1	coreset ablation	Planned
E-007	PatchCore	ShanghaiTech	1	backbone/layer ablation	Planned

ID	Model	Data	Scene(s)	Purpose	Status
E-008	AI-VAD	ShanghaiTech	1–3	video-native baseline	Planned
E-009	FUVAS	ShanghaiTech	1–3	few-shot curve	Planned
E-010	WinCLIP	ShanghaiTech	1–3	zero-shot	Planned
E-011	all	ShanghaiTech	1–13	full benchmark	Planned
E-012	best	Avenue	all	cross-dataset	Planned
E-013	all	ShanghaiTech	1–3	temporal ablation	Planned
E-014	best	video demo	n/a	Gradio + ONNX FPS	Planned

---

7. Memory and throughput notes

PatchCore memory model

```
bytes ≈ frames × patches_per_frame × feature_dim × 4
```

For 256×256 and WRN50 layer2+3:

- patches_per_frame ≈ 32 × 32 = 1,024
- feature_dim = 512 + 1,024 = 1,536
- one frame ≈ 6.0 MB
- 700 frames ≈ 4.2 GB
- 2,000 frames ≈ 12 GB (likely unsafe on 7.9 GB RAM)
- 69,976 frames ≈ 440 GB (impossible)

This is why the memory bank is sampled and why 2,000 frames may not be feasible without further reduction (lower resolution, fewer patches, or a different backbone).

Test metric memory

Pixel AUROC accumulates one value per pixel per frame:

- 256×256 = 65,536 values per frame
- 11,894 frames ≈ 780 million values
- float32 ≈ 3.1 GB raw, plus sort overhead ≈ 6.2 GB (observed allocation)
- 1,204 frames ≈ 79 million values ≈ 0.3–0.8 GB (worked)

Throughput observed

Stage	Rate
PatchCore fit (feature extraction + coreset)	~3 frames/s effective with 47 batches
Test inference + metrics, workers=0	~3.2 frames/s
Full-test inference only, before OOM	~5.9 frames/s

Workers were 0. Trying 2–3 workers is a planned optimisation, but RAM is tight.

---

8. Reporting requirements

Every final number must be accompanied by:

- model and backbone,
- feature layers,
- coreset ratio,
- memory-bank sampling rule,
- test sampling rule,
- input resolution,
- batch size and workers,
- threshold source,
- package versions,
- whether the visualizer was disabled,
- seed.

Without this, the numbers are not reproducible or comparable.